



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

数据安全实验报告

Lab 3: 秘密共享实践

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：刘哲理

2026 年 5 月 15 日

目录

一、 实验要求	2
二、 实验原理	2
(一) 秘密共享	2
1. 形式化定义	2
2. 分类	2
3. 门限秘密共享	3
(二) Shamir 方案	3
1. 基本思路	3
2. 秘密分配算法	4
3. 秘密重构算法	5
4. 重构方法一：解方程法	5
5. 重构方法二：多项式插值法	6
6. 同态性	7
三、 实验过程	9
(一) 协议设计	9
1. 角色设置	9
2. 参数设置	9
(二) 协议过程	9
(三) 代码编写	10
(四) 编译运行	14
四、 实验总结	16

一、实验要求

借鉴实验 3.2, 实现三个人对于他们拥有的数据的平均值的计算。

二、实验原理

(一) 秘密共享

秘密共享 (Secret Sharing) 是一种**将秘密分割存储**的密码技术。其目标是: 将一个秘密信息按照一定规则分割成**若干份“秘密份额 (Share)”**, 分别分发给不同参与者, 使得单独少量份额无法获得秘密, 而只有**满足条件的参与者集合才能恢复原始秘密**。其目的是阻止秘密过于集中, 以达到分散风险和容忍入侵的目的, 是信息安全和数据保密中的重要手段。

目前, 秘密共享已成为一种重要密码学工具, 在诸多多方安全计算协议中被使用, 例如拜占庭协议、多方隐私集合求交协议、阈值密码学等。

1. 形式化定义

令 D 为秘密值所在域, 令 D_1 为秘密份额所在域。令 $Shr : D \rightarrow D_1^n$ 为秘密分配算法 (可能是随机性算法), $Rec : D_1^k \rightarrow D$ 为秘密重构算法, 其中 D_1^n 表示 n 个秘密份额、 D_1^k 表示 k 个秘密份额。 (t, n) -秘密分享方案包含一对算法 (Shr, Rec) , 满足下述两个性质:

- **正确性**。令 $(s_1, s_2, \dots, s_n) = Shr(s)$, 则有:

$$\Pr[\forall k \geq t, Rec(s_{i_1}, \dots, s_{i_k}) = s] = 1$$

- **完美隐私性**。任意包含少于 t 个秘密份额的集合都不会在信息论层面上泄漏与秘密值相关的任何信息。

2. 分类

基于秘密分配和运算的形式, 秘密共享可分为基于位运算的加性秘密共享和基于线性代数的线性秘密共享。

基于秘密重构的条件或分享的份额数量等, 秘密共享又可分为:

- 门限秘密共享: 任意大于等于阈值的参与方集合可重构出秘密。
- 多重秘密共享: 参与方的子秘密可以多次使用, 分别恢复多个共享秘密
- 多秘密共享: 一次共享, 共享多个秘密, 且子秘密可以重复使用
- 可验证秘密共享: 可通过公共变量验证自己子秘密的正确性
- 动态秘密共享: 允许添加或删除参与方, 定期或不定期更新参与方的子秘密, 还允许在不同的时间恢复不同的秘密。

本实验中主要是使用门限秘密共享的方式。

3. 门限秘密共享

设 t 和 n 为两个正整数, 且 $t \leq n$, n 个需要共享秘密的参与者集合为 $P = \{P_1, \dots, P_n\}$ 。一个 (t, n) 门限秘密共享体制是指: 假设 $P_1, \dots, P_n$ 要共享同一个秘密 s , s 称为主秘密, 则至少 t 个参与者才可以共同恢复主秘密 s 。

秘密管理中心 P_0 负责对 s 进行管理和分配, P_0 掌握有秘密分配算法和秘密重构算法, 这两个算法均满足重构要求和安全性要求。

秘密分配: 秘密管理中心 P_0 首先将主秘密 s 输入秘密分配算法, 生成 n 个值, 分别为 $s_1, \dots, s_n$, 称 $s_1, \dots, s_n$ 为子秘密。然后秘密管理中心 P_0 分别将秘密分配算法产生的子秘密 $s_1, \dots, s_n$ 通过 P_0 与 P_i 之间的安全通信信道秘密地传送给参与者 P_i , 参与者 P_i 不得向任何人泄露自己所收到的子秘密 s_i 。

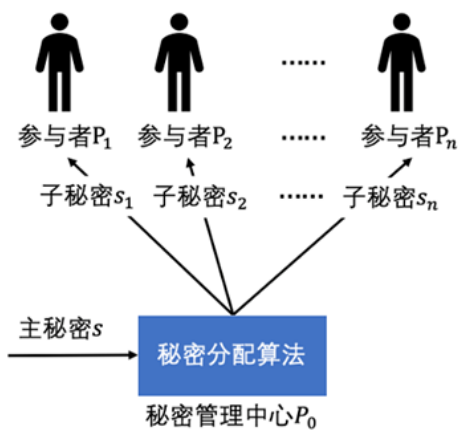


图 1: 秘密分配示意图

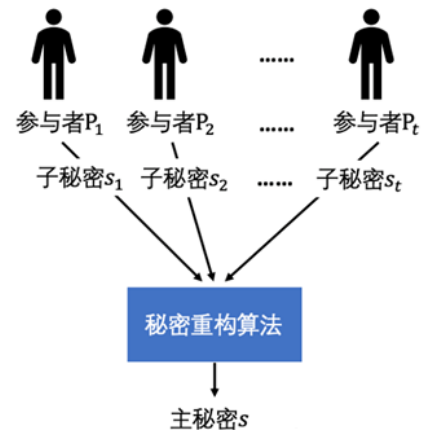


图 2: 秘密重构示意图

秘密重构: 门限值 t 指的是任意大于或等于 t 个参与者 P_i , 将各自掌握的子秘密 s_i 进行共享, 任意的一个参与者 P_i 在获得其余 $t-1$ 个参与者所掌握的子秘密后, 都能够独立地通过秘密重构算法恢复出主秘密 s 。即使有任意的 $n-t$ 个参与者丢失了各自所掌握的子秘密, 剩下 t 个参与者依旧可以通过将各自掌握的子秘密与其他参与者共享, 再使用秘密重构算法来重构出主秘密 s 。安全性要求任意攻击者通过收买等手段获取了少于 t 个的子秘密, 或者任意少于 t 个参与者串通都无法恢复出主秘密 s , 也无法得到主秘密 s 的信息。

(二) Shamir 方案

Shamir 秘密共享方案由密码学家 Adi Shamir 于 1979 年提出, 是最经典的门限秘密共享方案之一, 通常称为 “ (t, n) 门限秘密共享方案”

1. 基本思路

Shamir 方案的核心数学基础是: 在有限域中, 任意 t 个点可以唯一确定一个次数不超过 $t-1$ 的多项式。

Shamir 方案将秘密 S 放在多项式的常数项中, 构造如下多项式:

$$f(x) = S + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$$

其中:

- S : 秘密;
- $a_1, a_2, \dots, a_{t-1}$: 是随机选择的系数;
- $f(0) = S$, 即多项式在 $x = 0$ 处的值就是秘密。

然后给每个参与者一个点:

$$(x_i, f(x_i))$$

每一个点就是一个秘密份额。

只要获得 t 个点, 就可以恢复整个多项式, 从而计算:

$$S = f(0)$$

但如果只有少于 t 个点, 则无法唯一确定该多项式, 也就无法确定 S 。

Shamir 方案通常不是在普通实数范围内计算, 而是在有限域 F_p 中计算。

一般选择一个大素数 p , 要求:

$$p > S, \quad p > n$$

所有运算都在模 p 下进行。

也就是说:

$$f(x) = S + a_1x + \dots + a_{t-1}x^{t-1} \pmod{p}$$

这样做是为了:

- 保证安全性: 有限域上的随机系数可以提供信息论安全。
- 避免小数和误差: 所有计算都是整数模运算, 没有浮点误差。
- 便于计算逆元: 插值和解方程时需要除法, 在有限域中除法通过乘法逆元实现。

2. 秘密分配算法

Shamir 秘密共享的分配过程由一个秘密分发者 Dealer 完成。

输入包括:

- 秘密 S ;
- 参与者数量 n ;
- 门限值 t ;
- 一个素数 p , 满足 $p > S, p > n$ 。

先构造随机多项式。首先随机选择 $t-1$ 个系数:

$$a_1, a_2, \dots, a_{t-1} \in \mathbb{F}_p$$

构造 $(t-1)$ 次多项式:

$$f(x) = S + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

其中常数项就是秘密:

$$f(0) = S$$

多项式构造完成后, 生成秘密份额。选择 n 个不同的非零横坐标:

$$x_1, x_2, \dots, x_n$$

通常可以简单取:

$$x_i = i, \quad i = 1, 2, \dots, n$$

然后计算:

$$y_i = f(x_i) \pmod{p}$$

于是每个参与者 P_i 得到一个份额:

$$Share_i = (x_i, y_i)$$

注意:

- x_i 可以公开;
- y_i 必须保密;
- 不同参与者的 x_i 不能相同;
- 不能使用 $x = 0$, 因为 $f(0)$ 就是秘密本身。

3. 秘密重构算法

秘密重构的目标是: 根据任意 t 个秘密份额恢复出 $f(0)$, 也就是秘密 S 。

假设我们拿到 t 个份额:

$$(x_1, y_1), (x_2, y_2), \dots, (x_t, y_t)$$

由于这些点都在同一个 $t-1$ 次多项式上, 因此可以通过这些点恢复多项式 $f(x)$, 再计算:

$$S = f(0)$$

常见重构方法有两类:

1. 解方程法;
2. 多项式插值法, 尤其是拉格朗日插值法。

4. 重构方法一: 解方程法

设门限值为 t , 则多项式次数为 $t-1$ 。多项式形式为:

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

其中:

$$a_0 = S$$

如果拿到 (t) 个份额, 就可以建立 (t) 个方程:

$$\begin{cases} y_1 = a_0 + a_1x_1 + \dots + a_{t-1}x_1^{t-1} \\ y_2 = a_0 + a_1x_2 + \dots + a_{t-1}x_2^{t-1} \\ \vdots \\ y_t = a_0 + a_1x_t + \dots + a_{t-1}x_t^{t-1} \end{cases} \pmod{p}$$

这是一个 t 元一次方程组，未知数是：

$$a_0, a_1, \dots, a_{t-1}$$

解出 a_0 后，就得到秘密：

$$S = a_0$$

5. 重构方法二：多项式插值法

解方程法虽然直观，但在实际算法中，更常用的是多项式插值法，尤其是拉格朗日插值。先介绍拉格朗日插值基本思想。

已知 t 个点：

$$(x_1, y_1), (x_2, y_2), \dots, (x_t, y_t)$$

可以构造多项式：

$$f(x) = \sum_{i=1}^t y_i \cdot \ell_i(x), \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

其中：

$$\ell_i(x)$$

称为拉格朗日基多项式。它满足：

$$\ell_i(x_i) = 1$$

并且当 $j \neq i$ 时：

$$\ell_i(x_j) = 0$$

因此：

$$f(x_i) = y_i$$

在 Shamir 方案中，我们并不一定需要恢复整个多项式。因为秘密是：

$$S = f(0)$$

所以只需要计算：

$$f(0) = \sum_{i=1}^t y_i \ell_i(0)$$

其中：

$$\ell_i(0) = \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j}$$

即：

$$S = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

这个公式是 Shamir 秘密重构中最常用的形式。

6. 同态性

Shamir 秘密共享不仅可以用于保存秘密，还具有重要的同态性质。所谓同态性，是指：**可以直接对秘密份额进行某些运算，而这些运算对应于原始秘密上的运算**。也就是说，不恢复秘密，也可以在份额层面完成计算。——这是 Shamir 方案成为多方安全计算 MPC 基础的重要原因。

加法同态性：

假设有两个秘密：

$$S_A, S_B$$

分别用 Shamir 方案共享。构造两个多项式：

$$\begin{aligned} f_A(x) &= S_A + a_1x + \cdots + a_{t-1}x^{t-1} \\ f_B(x) &= S_B + b_1x + \cdots + b_{t-1}x^{t-1} \end{aligned}$$

其中：

$$\begin{aligned} f_A(0) &= S_A \\ f_B(0) &= S_B \end{aligned}$$

参与者 P_i 持有：

$$f_A(x_i), f_B(x_i)$$

现在每个参与者本地相加：

$$h(x_i) = f_A(x_i) + f_B(x_i)$$

则这些新份额来自多项式：

$$h(x) = f_A(x) + f_B(x)$$

并且：

$$h(0) = f_A(0) + f_B(0) = S_A + S_B$$

因此：各参与者只需要把自己的两个份额相加，就得到秘密 $S_A + S_B$ 的合法份额。写作公式就是——如果：

$$\begin{aligned} \text{Share}_i(S_A) &= (x_i, y_i^A) \\ \text{Share}_i(S_B) &= (x_i, y_i^B) \end{aligned}$$

则：

$$\text{Share}_i(S_A + S_B) = (x_i, y_i^A + y_i^B \pmod{p})$$

恢复后得到：

$$S_A + S_B \pmod{p}$$

乘法同态性：

假设有两个秘密 (S_A, S_B) ，对应多项式分别为：

$$f_A(x), f_B(x)$$

其中：

$$f_A(0) = S_A, f_B(0) = S_B$$

每个参与者本地相乘：

$$z_i = f_A(x_i) \cdot f_B(x_i)$$

这些点对应于新多项式：

$$g(x) = f_A(x) \cdot f_B(x)$$

并且：

$$g(0) = f_A(0) \cdot f_B(0) = S_A S_B$$

因此，本地乘法在语义上对应秘密的乘法。然而，多项式次数升高时会产生问题。如果：

$$f_A(x), f_B(x)$$

都是 $(t-1)$ 次多项式，那么乘积：

$$g(x) = f_A(x)f_B(x)$$

的次数最高为：

$$2(t-1)$$

也就是：

$$2t-2$$

这意味着：

- 原来只需要 t 个份额即可恢复；
- 乘法后需要 $2t-1$ 个份额才能恢复；
- 门限结构发生了变化。

所以，Shamir 的乘法同态不是像加法那样“直接保持门限不变”。综上，本地乘法可以得到：

$$S_A S_B$$

的共享形式，但共享多项式次数变高。因此通常需要额外操作：

- 降阶；
- 重新随机化；
- 交互式协议；
- Beaver 三元组；
- MPC 中的乘法门协议。

总结：

表 1: Shamir 方案同态性质总结

运算	份额层操作	秘密层效果	是否保持原门限
加法	$(y_i^A + y_i^B)$	$(S_A + S_B)$	是
减法	$(y_i^A - y_i^B)$	$(S_A - S_B)$	是
常数乘法	(cy_i)	(cS)	是
乘法	$(y_i^A y_i^B)$	$(S_A S_B)$	否，次数升高

- Shamir 方案天然支持线性运算；
- 对非线性运算，尤其是乘法，需要额外协议；
- 这正是它在 MPC 中既强大又需要复杂协议设计的原因。

三、 实验过程

(一) 协议设计

实验要解决的问题是——三个参与方 Student_1、Student_2 和 Student_3 分别拥有隐私数据 a 、 b 和 c 。希望在不公开各自原始数据的情况下，计算三个数据的平均值：

$$average = (a + b + c)/3$$

参考教材实验 4.2 的投票统计过程，基于 Shamir (2,3) 门限秘密共享完成隐私求和，再由结果方计算平均值。

1. 角色设置

1. 数据方：Student_1、Student_2、Student_3，分别持有隐私输入 a 、 b 、 c 。
2. 计算方：Student_1、Student_2、Student_3，各自保存并计算自己收到的秘密份额。
3. 结果方：average_counter，可以从三个计算方中任选两个计算结果份额，恢复最终求和结果并计算平均值。

2. 参数设置

1. 门限：Shamir (2,3)（三人中至少两人才可恢复秘密）
2. 模数： $p = 1000000007$ 。
3. 参与方的函数横坐标： $x_1 = 1$, $x_2 = 2$, $x_3 = 3$ 。
4. 输入数据要求：三个输入数据应为非负整数，且 $a + b + c < p$ ，避免模 p 运算发生回绕。

(二) 协议过程

1. Student_1 将自己的隐私数据 a 作为多项式 $f_1(x)$ 的常数项，随机生成一次多项式：

$$f_1(x) = a + r_1 * x$$

计算 $f_1(1)$ 、 $f_1(2)$ 、 $f_1(3)$ ，分别作为 a_1 、 a_2 、 a_3 。

2. Student_2 将自己的隐私数据 b 作为多项式 $f_2(x)$ 的常数项，随机生成一次多项式：

$$f_2(x) = b + r_2 * x$$

计算 $f_2(1)$ 、 $f_2(2)$ 、 $f_2(3)$ ，分别作为 b_1 、 b_2 、 b_3 。

3. Student_3 将自己的隐私数据 c 作为多项式 $f_3(x)$ 的常数项，随机生成一次多项式：

$$f_3(x) = c + r_3 * x$$

计算 $f_3(1)$ 、 $f_3(2)$ 、 $f_3(3)$ ，分别作为 c_1 、 c_2 、 c_3 。

4. 三个学生按照下列方式获得属于自己的秘密份额：

Student_1 获得 a_1 、 b_1 、 c_1 ；Student_2 获得 a_2 、 b_2 、 c_2 ；Student_3 获得 a_3 、 b_3 、 c_3 。

5. 每个学生在本地将收到的三个份额相加：

$$d_1 = a_1 + b_1 + c_1 \mod p$$

$$d_2 = a_2 + b_2 + c_2 \mod p$$

$$d_3 = a_3 + b_3 + c_3 \mod p$$

由于 Shamir 秘密共享具有加法同态性， d_1 、 d_2 、 d_3 正好是秘密 $d = a + b + c$ 的三个 Shamir 份额。

6. 结果方任选两个计算结果份额，例如 d_2 和 d_3 ，通过拉格朗日插值重构：

$$d = a + b + c$$

7. 结果方计算平均值：

$$average = d/3$$

如果 d 不能被 3 整除，程序会同时输出分数形式和小数形式。

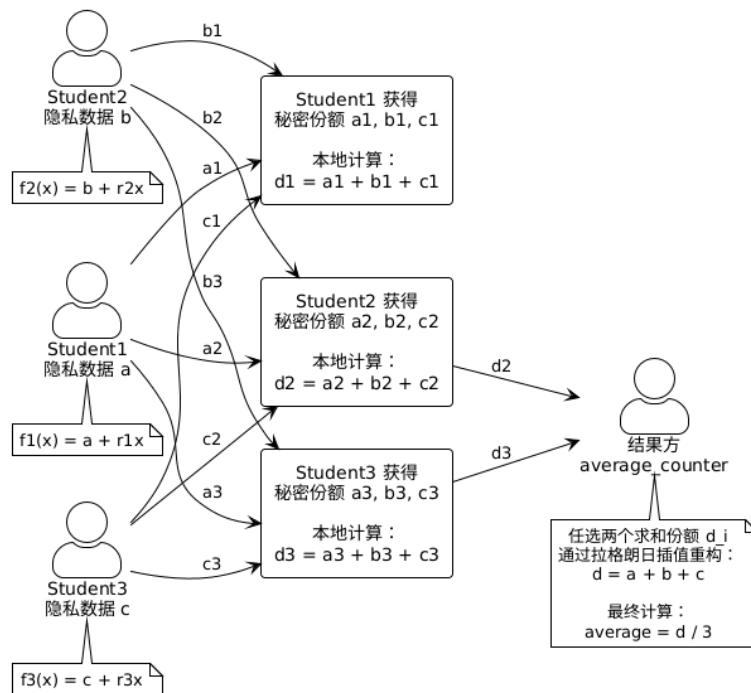


图 3：三人数据平均值计算协议流程

(三) 代码编写

参考教材中例题来实现，基本代码架构保持不变，如下四个文件：

1. `ss_function.py`: Shamir 秘密共享基础函数，包括快速幂、多项式生成、多项式求值、拉格朗日重构。

2. ss_student.py: 每个学生运行一次, 输入自己的 id 和隐私数据, 生成并保存秘密份额。
3. count_student.py: 每个学生运行一次, 读取属于自己的三个秘密份额并求和。
4. average_counter.py: 结果方读取任意两个求和份额, 恢复总和并输出平均值。

首先,ss_function.py 基本保持教材原代码逻辑不变,仍包含 quickpower、get_polynomial、count_polynomial、restructure_polynomial 四个函数。只做了代码格式整理, 函数功能和调用方式没有改变:

```
1 import random
2
3 # 快速幂计算 $a^b \pmod p$ 
4 def quickpower(a, b, p):
5     a = a % p
6     ans = 1
7     while b != 0:
8         if b & 1:
9             ans = (ans * a) % p
10            b >>= 1
11            a = (a * a) % p
12    return ans
13
14
15 # 构建多项式: x0为常数项系数, T为最高次项次数, p为模数, fname为多项式名
16 def get_polynomial(x0, T, p, fname):
17     f = []
18     f.append(x0)
19     for i in range(0, T):
20         f.append(random.randrange(0, p))
21     # 输出多项式
22     f_print = "f" + fname + "=" + str(f[0])
23     for i in range(1, T + 1):
24         f_print += "+" + str(f[i]) + "x^" + str(i)
25     print(f_print)
26     return f
27
28
29 # 计算多项式值
30 def count_polynomial(f, x, p):
31     ans = f[0]
32     for i in range(1, len(f)):
33         ans = (ans + f[i] * quickpower(x, i, p)) % p
34     return ans
35
36
37 # 重构函数f并返回f(0)
38 def restructure_polynomial(x, fx, t, p):
39     ans = 0
40     # 利用多项式插值法计算出x=0时多项式的值
```

```

41     for i in range(0, t):
42         fx[i] = fx[i] % p
43         fxi = 1
44         # 在模p下, (a/b)%p=(a*c)%p, 其中c为b在模p下的逆元, c=b^(p-2)%p
45         for j in range(0, t):
46             if j != i:
47                 fxi = (-1 * fxi * x[j] * quickpower(x[i] - x[j], p - 2, p)) % p
48             fxi = (fxi * fx[i]) % p
49             ans = (ans + fxi) % p
50     return ans

```

ss_student.py 的框架也保持与教材一致, 仍然输入参与方 id 和秘密值 s, 生成 Shamir (2,3) 秘密份额。将 console 输入提示语从“投票值”改成了“数据”, 表示这里的秘密不再是 0/1 投票, 而是每个学生拥有的待求平均值的数据即可:

```

1  import ss_function as ss_f
2
3  # 设置模数p
4  p = 1000000007
5  print(f"模数p: {p}")
6
7  # 输入参与方id以及秘密数据s
8  id = int(input("请输入参与方id:"))
9  s = int(input(f"请输入student_{id}的数据s:"))
10
11 # 秘密份额为(share_x,share_y)
12 shares_x = [1, 2, 3]
13 shares_y = []
14
15 # 计算多项式及秘密份额(t=2,n=3)
16 print(f"Student_{id}的数据的多项式及秘密份额: ")
17 f = ss_f.get_polynomial(s, 1, p, str(id))
18 temp = []
19 for j in range(0, 3):
20     temp.append(ss_f.count_polynomial(f, shares_x[j], p))
21     print(f"({shares_x[j]},{temp[j]}")
22     shares_y.append(temp[j])
23
24 # Student_id将自己的数据的秘密份额分享给另外两个学生
25 # 将三份秘密份额分别保存到student_id_1.txt,student_id_2.txt,student_id_3.txt
26 # Student_i获得Student_id_i.txt
27 for i in range(1, 4):
28     with open(f"student_{id}_{i}.txt", "w") as f:
29         f.write(str(shares_y[i - 1]))

```

count_student.py 与教材基本一致, 仍然读取 student_1_id.txt、student_2_id.txt、student_3_id.txt, 并计算 $d_{id} = a_{id} + b_{id} + c_{id} \bmod p$ 。新增了一行输出, 用来显示当前学生计算得到的求和秘密份额, 便于实验过程中的观察:

```

1  p = 1000000007
2
3  # 输入参与方id
4  id = int(input("请输入参与方id:"))
5
6  # Student_id读取属于自己的秘密份额student_1_id.txt,student_2_id.txt,student_3_id.txt
7  data = []
8  for i in range(1, 4):
9      with open(f"student_{i}_{id}.txt", "r") as f:
10         data.append(int(f.read()))
11
12 # 计算三个秘密份额的和
13 d = 0
14 for i in range(0, 3):
15     d = (d + data[i]) % p
16
17 # 将求和后的秘密份额保存到文件d_id.txt内
18 with open(f"d_{id}.txt", "w") as f:
19     f.write(str(d))
20
21 print(f"Student_{id}的求和秘密份额为: {d}")

```

average_counter.py 对应教材中的 vote_counter.py。教材中重构后直接输出“得票结果”，本实验中重构得到 $d = a + b + c$ 后，还要计算 $average = d / 3$ 。为了处理不能整除的情况，代码 import 了 Fraction 包，可以实现输出分数形式和小数近似值：

```

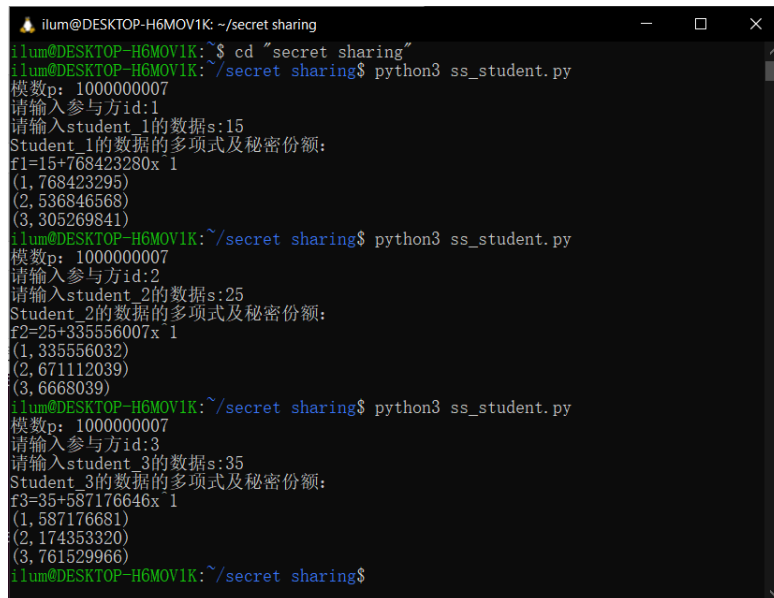
1  from fractions import Fraction
2  import ss_function as ss_f
3
4  # 设置模数p
5  p = 1000000007
6
7  # 随机选取两个参与方，例如student2和student3，获得d2,d3，从而恢复出d=a+b+c
8  # 读取d2,d3
9  d_23 = []
10 for i in range(2, 4):
11     with open(f"d_{i}.txt", "r") as f:
12         d_23.append(int(f.read()))
13
14 # 加法重构获得d
15 d = ss_f.restructure_polynomial([2, 3], d_23, 2, p)
16
17 # 计算平均值average=d/3
18 average = Fraction(d, 3)
19 print(f"三个数据的和为: {d}")
20 if average.denominator == 1:
21     print(f"三个数据的平均值为: {average.numerator}")
22 else:
23     print(f"三个数据的平均值为: {average}, 约为: {float(average)}")

```

(四) 编译运行

由于实验过程中直接手动新建文件并写入了代码，不再需要像指导教材中一样用 `touch` 命令去新建文件并写入。

测试时，先对每个学生运行一次 `ss_student.py`，输入学生 `id` 和他们各自持有的数据：



```
ilum@DESKTOP-H6MOV1K: ~/secret sharing
ilum@DESKTOP-H6MOV1K:~$ cd "secret sharing"
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:1
请输入student_1的数据s:15
Student_1的数据的多项式及秘密份额:
f1=15+768423280x^1
(1, 768423295)
(2, 536846568)
(3, 305269841)
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:2
请输入student_2的数据s:25
Student_2的数据的多项式及秘密份额:
f2=25+335556007x^1
(1, 335556032)
(2, 671112039)
(3, 6668039)
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:3
请输入student_3的数据s:35
Student_3的数据的多项式及秘密份额:
f3=35+587176646x^1
(1, 587176681)
(2, 174353320)
(3, 761529966)
ilum@DESKTOP-H6MOV1K:~/secret sharing$
```

图 4: 运行 `ss_student.py` 输入学生 `id` 和数据

在当前目录下会产生 9 个 `.txt` 文件，分别保存三个秘密值的秘密份额：

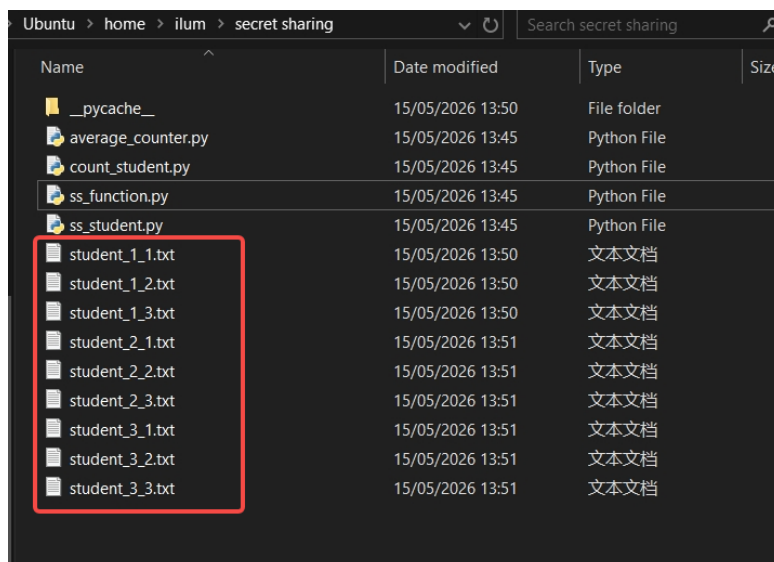
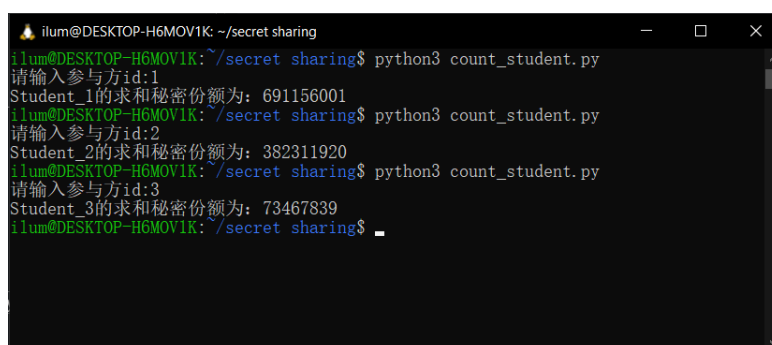


图 5: 生成的秘密份额文件

而后，再对每个学生运行一次 `count_student.py`，读取属于自己的三个秘密份额并求和：



```
ilum@DESKTOP-H6MOV1K: ~/secret sharing
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:1
Student_1的求和秘密份额为: 691156001
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:2
Student_2的求和秘密份额为: 382311920
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:3
Student_3的求和秘密份额为: 73467839
ilum@DESKTOP-H6MOV1K:~/secret sharing$
```

图 6: 运行 count_student.py 读取并求和秘密份额

求和结果输出为相应编号的 .txt 文件中:

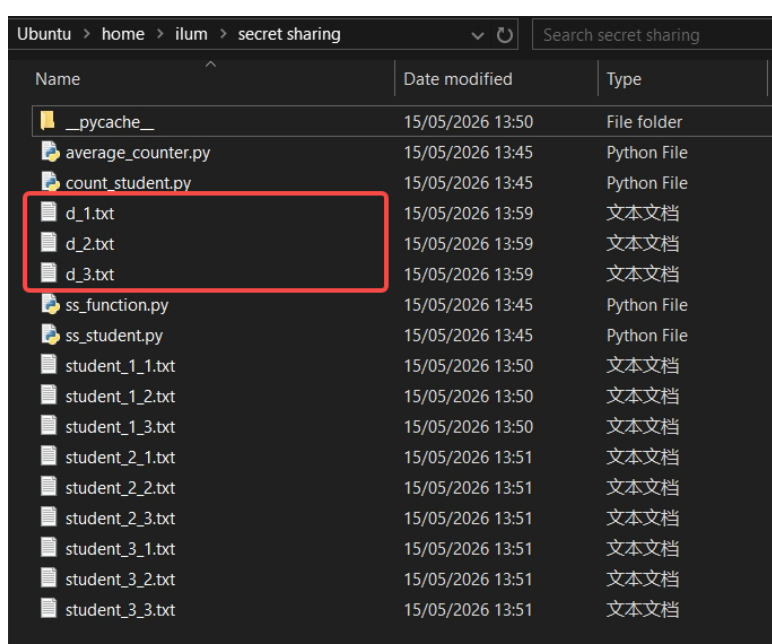
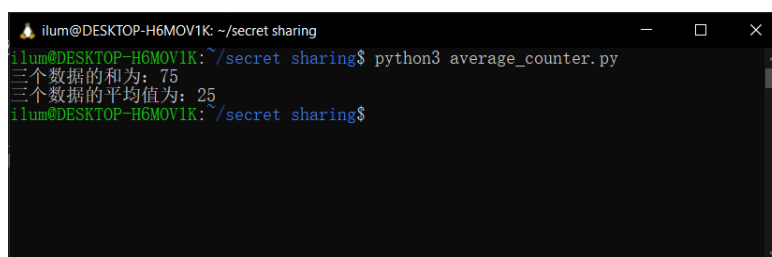


图 7: 求和结果份额文件

最后执行 average_counter.py, 结果方随机读取两个求和份额, 恢复总和并输出平均值:

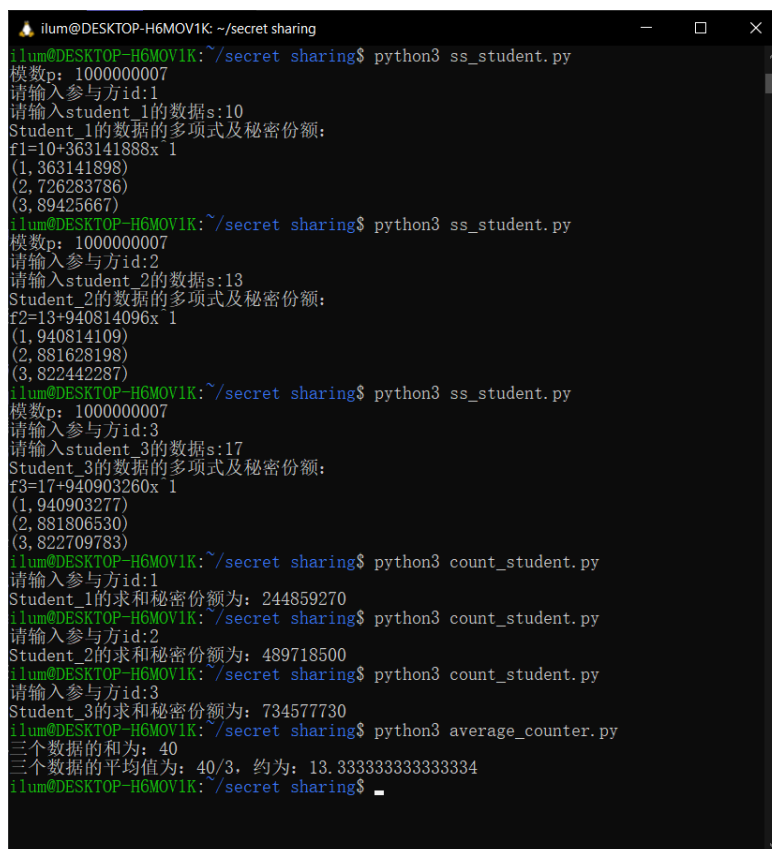


```
ilum@DESKTOP-H6MOV1K: ~/secret sharing
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 average_counter.py
三个数据的和为: 75
三个数据的平均值为: 25
ilum@DESKTOP-H6MOV1K:~/secret sharing$
```

图 8: 运行 average_counter.py 输出平均值

在上面的测试样例中, 三个数分别是 15, 25, 35, 求和为 75, 平均数为 25, 恢复结果准确无误。

为了测试引入的 Fraction 功能, 随机选择一组求平均值不为整数的数据进行一次完整测试:

A terminal window titled 'ilum@DESKTOP-H6MOV1K: ~/secret sharing' showing the execution of several Python scripts. The scripts use the Fraction module to perform calculations involving modular arithmetic and polynomial interpolation. The output shows the results of these calculations, including the reconstruction of a secret 's' from three shares.

```
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:1
请输入student_1的数据s:10
Student_1的数据的多项式及秘密份额:
f1=10+363141888x^1
(1, 363141898)
(2, 726283786)
(3, 89425667)
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:2
请输入student_2的数据s:13
Student_2的数据的多项式及秘密份额:
f2=13+940814096x^1
(1, 940814109)
(2, 881628198)
(3, 822442287)
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 ss_student.py
模数p: 1000000007
请输入参与方id:3
请输入student_3的数据s:17
Student_3的数据的多项式及秘密份额:
f3=17+940903260x^1
(1, 940903277)
(2, 881806530)
(3, 822709783)
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:1
Student_1的求和秘密份额为: 244859270
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:2
Student_2的求和秘密份额为: 489718500
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 count_student.py
请输入参与方id:3
Student_3的求和秘密份额为: 734577730
ilum@DESKTOP-H6MOV1K:~/secret sharing$ python3 average_counter.py
三个数据的和为: 40
三个数据的平均值为: 40/3. 约为: 13.333333333333334
ilum@DESKTOP-H6MOV1K:~/secret sharing$
```

图 9: Fraction 功能测试结果

恢复后的结果也是正确的，并且能够输出精确的分数和近似的小数。

四、 实验总结

本实验实现了一个基于 Shamir 门限秘密共享的“三人数据平均值计算”协议，目标是在不直接公开三位参与者原始数据的情况下，计算出三人数据的平均值。实验整体框架参考教材实验 4.2，主要在于设计一个适合于实验题目的新的协议，代码改动较少。实际代码中添加了一些输出信息以及 Fraction 支持，使得过程和结果都更加清晰可靠。

本次实验的代码已经上传至 GitHub 仓库：[data-security/lab3](https://github.com/data-security/lab3)。